



Java If ... Else

[< Previous](#)[Next >](#)

Java Conditions and If Statements

Conditions and if statements let you control the flow of your program - deciding which code runs, and which code is skipped.

Think of it like real life: *If it rains, take an umbrella. Otherwise, do nothing.*

Every **if** statement needs a condition that results in **true** or **false**.

This means **if** statements work hand-in-hand with **boolean** values:

Example

[Get your own Java Server](#)

```
boolean isRaining = true;

if (isRaining) {
    System.out.println("Bring an umbrella!");
}
```

[Try it Yourself »](#)

Most often, conditions are created using comparison operators, like the ones below:

- Less than: **a < b**
- Less than or equal to: **a <= b**
- Greater than: **a > b**
- Greater than or equal to: **a >= b**
- Equal to: **a == b**



Java has the following conditional statements:

- Use **if** to specify a block of code to be executed, if a specified condition is true
- Use **else** to specify a block of code to be executed, if the same condition is false
- Use **else if** to specify a new condition to test, if the first condition is false
- Use **switch** to specify many alternative blocks of code to be executed

The if Statement

The **if** statement specifies a block of code to be executed if a condition is **true** :

Syntax

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

The *condition* inside the **if** statement must result in a **boolean** value - it can be either a boolean expression (like **x > y**) or a boolean variable (like **isLightOn**).

Also note that **if** is in lowercase letters. Uppercase letters (If or IF) will generate an error.

In the example below, we test two values to find out if 20 is greater than 18. If the condition is **true**, print some text:

Example

```
if (20 > 18) {  
    System.out.println("20 is greater than 18");  
}
```



You can also compare variables:

Example

```
int x = 20;
int y = 18;
if (x > y) {
    System.out.println("x is greater than y");
}
```

[Try it Yourself »](#)

Example explained

In the example above we use two variables, **x** and **y**, to test whether x is greater than y (using the **>** operator). As x is 20, and y is 18, and we know that 20 is greater than 18, we print to the screen that "x is greater than y".

Comparison is also often used to check if two values are *equal*, using the **==** operator:

Example

```
int x = 20;
int y = 20;
if (x == y) {
    System.out.println("x is equal to y");
}
```

[Try it Yourself »](#)

Here the condition **x == y** is true, because both **x** and **y** are 20, so the message "x is equal to y" is printed.



Using Boolean Variables

You can also test boolean variables directly in an **if** statement:

Example

```
boolean isLightOn = true;

if (isLightOn) {
    System.out.println("The light is on.");
}
```

[Try it Yourself »](#)

Note: Writing **if (isLightOn)** is the same as writing **if (isLightOn == true)**, but shorter and easier to read.

Here is the same example with the value **false** to see that the program continues even when the code block does not run:

Example

```
boolean isLightOn = false;

if (isLightOn) {
    System.out.println("The light is on."); // This will not be printed
}

System.out.println("This line runs no matter what, because it is outside the if block");
```

[Try it Yourself »](#)



If an **if** statement has only one line of code, you can write it without curly braces **{ }**:

Example

```
if (20 > 18)
    System.out.println("20 is greater than 18");
```

Try it Yourself »

Potential Problem

Without braces, only the **first line** after the **if** belongs to it. Any other lines will run no matter what, which can lead to unexpected results:

Example

```
int x = 20;
int y = 18;

if (x > y)
    System.out.println("x is greater than y"); // Belongs to if
    System.out.println("This line runs no matter what (not part of the if statement)");

// Output:
// x is greater than y
// This line runs no matter what (not part of the if statement)
```

Try it Yourself »

The Safe Way



Example

```
int x = 20;
int y = 18;

if (x > y) {
    System.out.println("x is greater than y");
    System.out.println("Both lines are part of the if");
}

// Some code outside if
System.out.println("I am outside if, not part of if!");
```

Try it Yourself »

Tip: Always using braces `{ }` makes your code clearer, easier to read, and prevents subtle bugs.

In the next chapters, you will also learn how to handle `else` (when the condition is false), `else if` (to test multiple conditions), and `switch` (to handle many possible values).

Exercise ?

Drag and drop the correct keyword and condition to make the code print
Good night.

```
int time = 20;
    if      ( time > 18 ) {
```

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Sign In](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[HOW TO](#)[W3.CSS](#)

time < 18

else

[Submit Answer »](#)[◀ Previous](#)[Sign in to track progress](#)[Next ▶](#)[REMOVE ADS](#)